

E-commerce Purchase Prediction at Scale

Big Data ML Pipeline on 42.4M Events

Albina Venera

2026

Team 9 | Innopolis University

The Cost of Cart Abandonment

The reality of online retail:

- Only **1.75%** of events convert
- **98.25%** of uniform marketing hits non-buyers
- Retargeting budget burns on noise

The opportunity:

Pay only where the signal is.

1.75%

purchase rate

98.25%

wasted reach under uniform marketing

Objective: Marketing Lift, Not Accuracy

Why Accuracy fails here

A trivial “never buys” classifier scores **98.25%** Accuracy and catches **zero buyers**.

The metric that matters:

$$\text{Lift}(t) = \frac{P(t)}{p} = \frac{\text{Precision at threshold } t}{\text{base purchase rate}}$$

Maximising **PR-AUC** integrates lift across all thresholds.

We optimise for **PR-AUC** and **Recall** – not Accuracy.

The Scale of the Problem

42.4 M

events

5.6 GB

raw CSV

9

attributes per event

The Imbalance Challenge

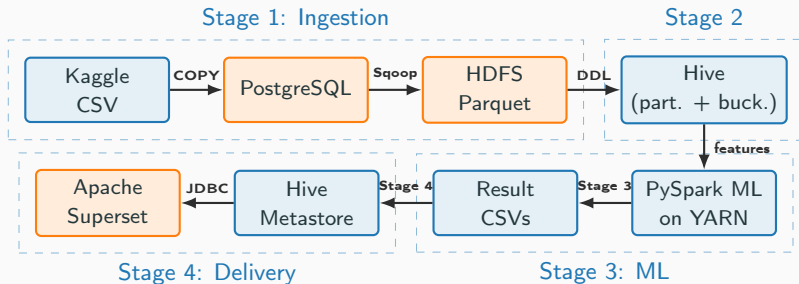
98.25% non-buyers

Green sliver = 1.75% actual purchases

Class ratio $\approx 56 : 1$

Implication: The minority class drives all the business value *and* all the modelling difficulty.

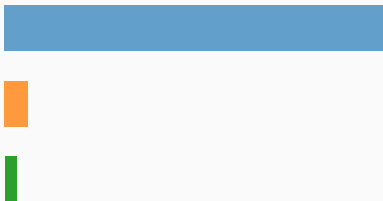
Big Data Architecture



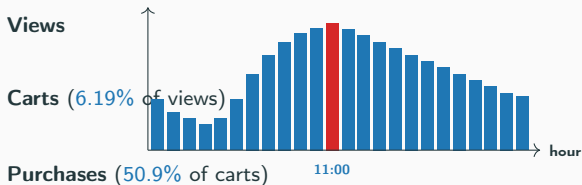
Hadoop YARN cluster (3 nodes, CentOS 7) | Hive metastore at `hadoop-02:9883`

What the Data Told Us

Session funnel



Hourly purchase distribution



Once a user adds to cart, they convert **half the time** – the cart is the high-signal moment.

Sharp peak at 11:00 UTC + “adjacent hour” (23↔0) effect ⇒ **cyclical** encoding required.

Custom CyclicalEncoder

`pyspark.ml.Transformer` subclass.

$$x_{\sin} = \sin\left(\frac{2\pi x}{p}\right), \quad x_{\cos} = \cos\left(\frac{2\pi x}{p}\right)$$

Applied to hour (p=24) and dayOfWeek (p=7).

⇒ Hour 23 and hour 0 sit next to each other in feature space, instead of 23× apart.

Cardinality control

Brand vocabulary > 10,000 values.

- Top **100 brands** retained
- Top **50 categories** retained
- All other values → "other"

Feature vector capped at ≈ 160 dims – no OOM, no information starvation. Computed from **training fold only** to prevent leakage.

Model Comparison on the Imbalanced Test Set

Model	PR-AUC	Recall	F1	Precision	ROC-AUC	Acc.
LR (Weighted)	0.0354	0.622	0.057	0.030	0.680	0.642
LR (Downsampled)	0.0354	0.622	0.057	0.030	0.680	0.641
RF (Weighted)	0.0350	0.529	0.059	0.031	0.669	0.704
RF (Downsampled)	0.0344	0.574	0.058	0.030	0.670	0.672

Verdict: [Weighted Logistic Regression](#) – best PR-AUC, best Recall, full training data.

Random Forest has higher Accuracy but misses 10pp more buyers – the wrong trade-off for marketing.

We Are $1.7\times$ Better Than Random

0.0354

our PR-AUC

0.0175

random baseline ($= p$)

$\Rightarrow$ **Lift** $\approx 2.0\times$ on ranking

At operating threshold $t = 0.5$:

- Precision = 3.00% vs. base 1.75%
- **Lift** $\approx 1.71\times$ per impression
- Recall = 62%

Marketing translation:

Reach **62% of buyers** by contacting only **36%** of users.

$\Rightarrow$ **42% cost reduction** for the same buyer coverage.

Risk Test: Price-Surge Simulation

Setup: Inflate every test-set price by +50%, re-score, measure Δ in predicted purchase probability.

Price	P_{base}	P_{surge}	Δ
\$575.79	0.495	0.447	-0.048
\$239.36	0.420	0.401	-0.019
\$308.63	0.194	0.179	-0.016
\$189.24	0.493	0.477	-0.016
\$200.52	0.254	0.242	-0.013
\$51.22	0.190	0.187	-0.003

Two robust findings:

- $\Delta < 0$ for every sample
⇒ **monotonic** price elasticity
- Magnitude scales with
baseline intent and absolute price

The model *understands* that raising prices reduces purchase intent, with high-intent expensive items most affected.

Production Challenges We Solved

HDFS Quota (32 GB cap)

Raw + Parquet + JSON splits exceeded budget.

Fix: Stage 2 deletes raw parquet on success;
replication factor tuned per dataset.

Spark OOM at 42M rows

Long-running CV jobs killed by executor evictions.

Fix: `.repartition(200)` + explicit
`.unpersist()` in finally blocks; tuned YARN
executor sizing.

JSON Bloating

Parquet→JSON expanded 7× on disk (losing
columnar compression).

Fix: Kept JSON splits minimal; binary formats
elsewhere.

What Comes Next

Real-time streaming

Migrate scoring path from batch CV to Spark Structured Streaming.

Score sessions *as they unfold* for in-flight retargeting.

Cost-sensitive learning

Replace uniform class weights with business-cost-aware loss.

Optimise expected revenue per impression, not symmetric error.

Additional avenues:

- Session-level aggregation features (path length, dwell time, repeat-visit)
- Gradient-boosted alternative (LightGBM via PySpark)
- Bayesian threshold optimisation against marketing cost matrix

Albina

ML & Evaluation

- Stage 2 Five business-insight EDA queries
- HDFS quota management
- Stage 3 PySpark ML pipeline
- Custom CyclicalEncoder transformer
- Balancing experiment (weights vs. downsample)
- Six-metric evaluation framework
- Price-surge risk simulation
- Pylint 10/10 refactor

Venera

Ingestion & Storage

- Stage 1 PostgreSQL + Sqoop pipeline
- Parquet vs. Avro benchmark
- Stage 2 partitioned + bucketed Hive table
- Stage 4 beeline / Superset wiring

Thank you.